

SCRIPTING Y PROGRAMACIÓN DE VIDEOJUEGOS

El Último Pilar

Análisis de Mecánica Principal y Arquitectura C#

Desacoplamiento por eventos · Adaptación en Unity · Diagrama de clases

Equipo de Scripting - Manulets y nacho 👍 - EXPERTOS EN C#.



Made with GAMMA

Arena Reactiva por Fases

La integridad del Pilar **altera dinámica e irreversiblemente** la topología y físicas del mapa en 4 fases progresivas. AURA

1

Fase 1

100% – 75%

Arena base intacta y despejada.

2

Fase 2

75% – 50%

Apertura del pozo central. Restricción espacial y riesgo de caída.

3

Fase 3

50% – 25%

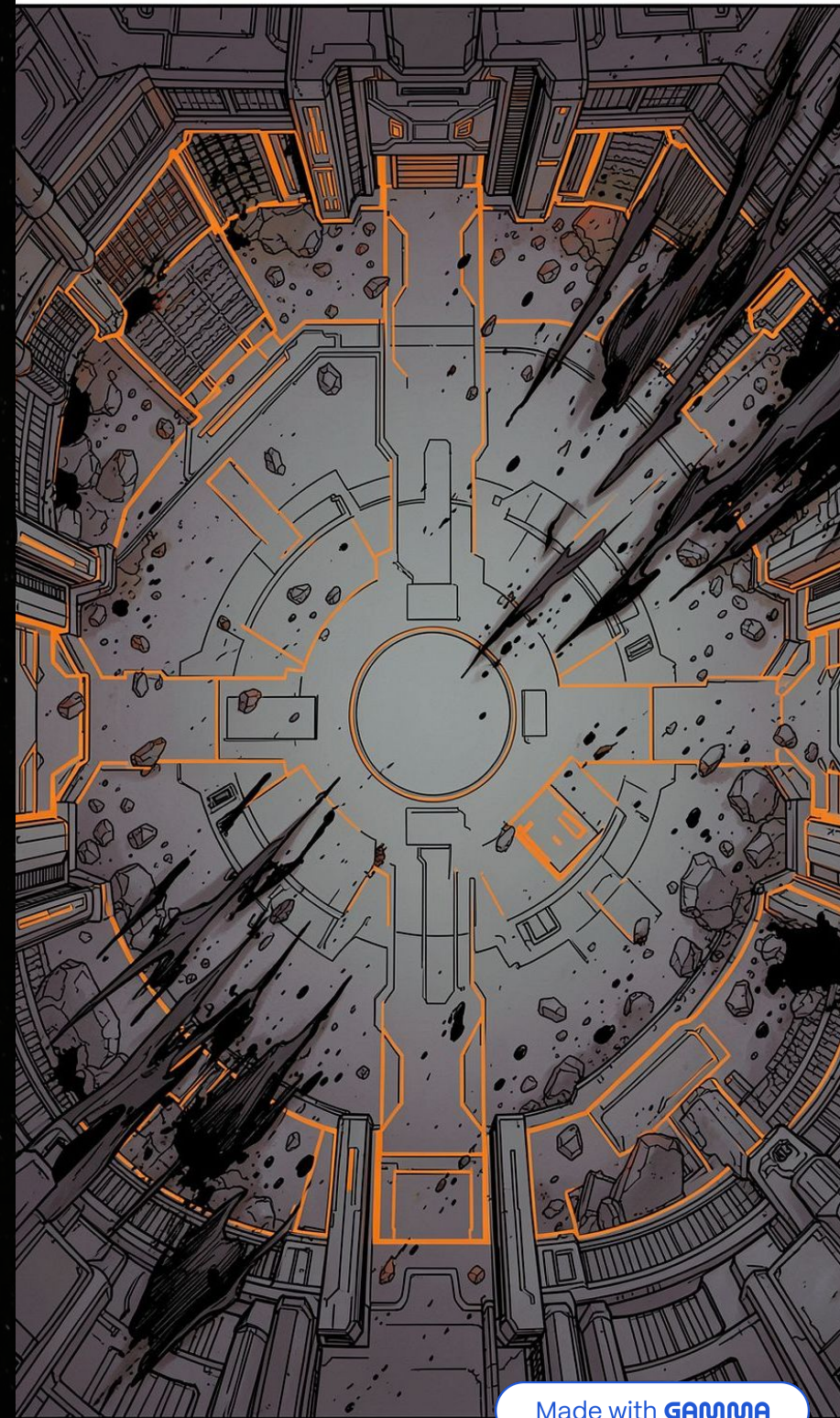
Zona de gravedad alterada. Control aéreo modificado.

4

Fase 4

< 25%

Protocolo de emergencia: escombros + torretas autónomas.



Pilar.cs — Adaptador de estado y fases

Rol arquitectónico

Cuida la salud del objetivo central y avisa cuando algo cambia, para que la arena, el HUD y el audio reaccionen cada uno por su cuenta.

Patrón Observador

Eventos nativos event Action
OnVidaCambiada, event Action
OnFaseCambiada y event Action
OnDañoRecibido para
comunicación desacoplada.

Transición acumulativa

En Update(),
StepToward(vidaActual) calcula
cada fase intermedia y
ChangePhase() las despacha una
por una si la salud cruza múltiples
umbrales en un solo frame.

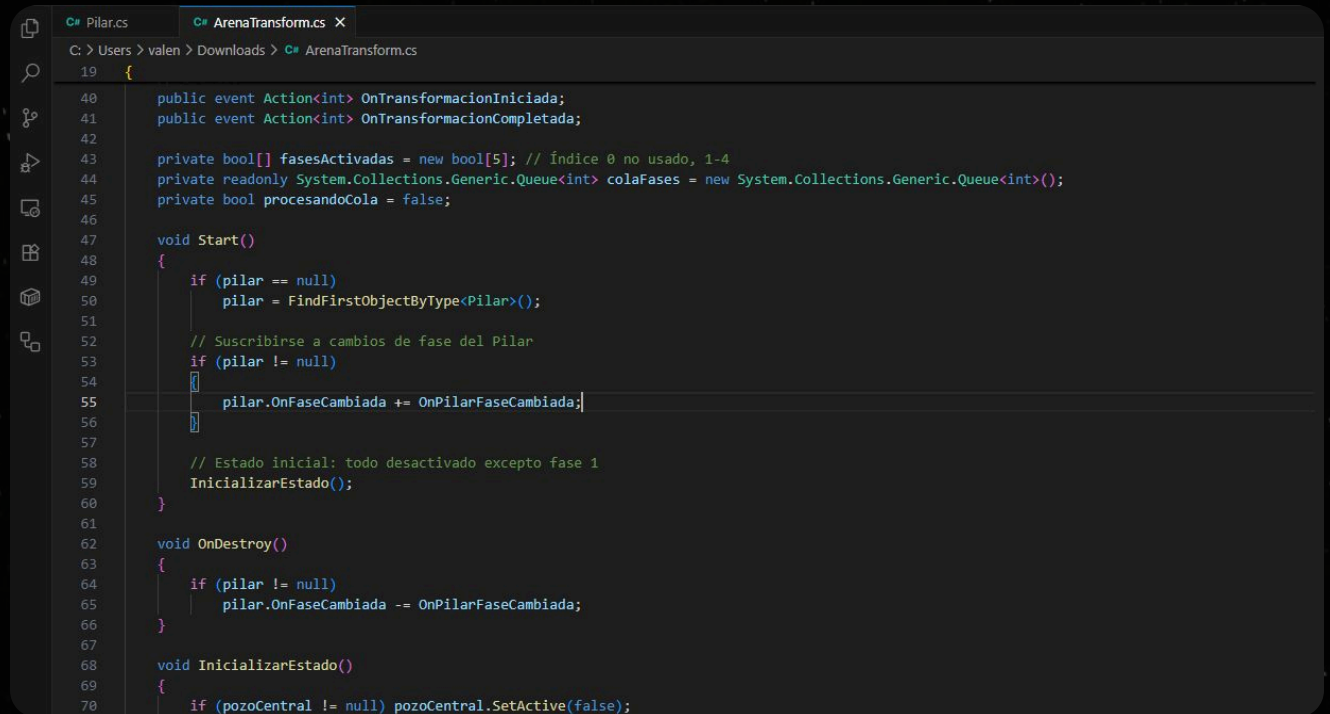
Defensa autónoma

ActivateTurrets() (privado, una sola vez por guard torretasActivas) delega en turretSpawner.Spawn(puntosTorretas, prefabTorreta) al entrar en Fase 4.

ArenaTransform.cs — Receptor y Adaptador de Entorno

Rol en el juego

Es el script que se encarga de escuchar al script pilar.cs y llamar a las funciones que se correspondan con las transformaciones de la arena en cada fase.



```
19 {
40 public event Action<int> OnTransformacionIniciada;
41 public event Action<int> OnTransformacionCompletada;
42
43 private bool[] fasesActivadas = new bool[5]; // índice 0 no usado, 1-4
44 private readonly System.Collections.Generic.Queue<int> colaFases = new System.Collections.Generic.Queue<int>();
45 private bool procesandoCola = false;
46
47 void Start()
48 {
49     if (pilar == null)
50         pilar = FindFirstObjectByType<Pilar>();
51
52     // Suscribirse a cambios de fase del Pilar
53     if (pilar != null)
54     {
55         pilar.OnFaseCambiada += OnPilarFaseCambiada;
56     }
57
58     // Estado inicial: todo desactivado excepto fase 1
59     InicializarEstado();
60 }
61
62 void OnDestroy()
63 {
64     if (pilar != null)
65         pilar.OnFaseCambiada -= OnPilarFaseCambiada;
66 }
67
68 void InicializarEstado()
69 {
70     if (pozoCentral != null) pozoCentral.SetActive(false);
```

Ciclo de vida limpio

Como todo el mapa, se crea en Start en la escena y se borra en onDestroy, para que no queden cosas cacheadas y sea mas ameno el uso del inspector.

Administracion

Aministra una cola para que las fases no se superpongan en el caso de que haya que pasar de a muchas fases.

Físicas en tiempo real

Todas sus fases corresponden a mecanicas que manipulan la fisca, el mapa y el ambiente de juego en general.

Diagrama de Clases y Separación de Capas

Capa de reglas (C# puro)

MatchFlow, MatchState,
PlayerRoster<T>, IDamageable –
independientes del motor.

Capa de adaptadores (Unity)

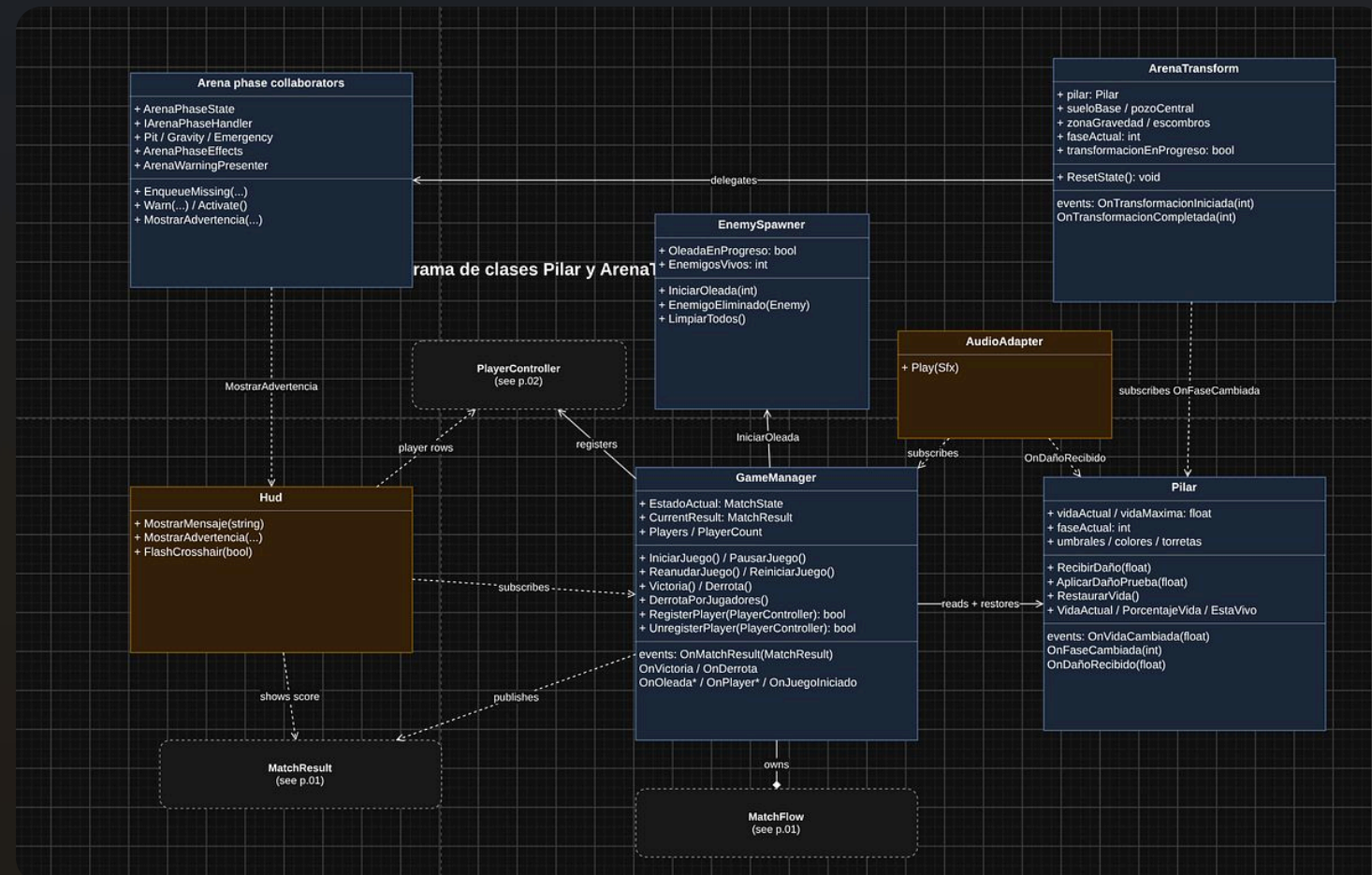
Pilar, ArenaTransform, GameManager — traducen reglas a comportamientos del motor.

Bajo acoplamiento

ArenaTransform no manipula la vida del Pilar; solo reacciona a sus eventos públicos.

Contratos de combate

Interfaz IDamageable y objeto inmutable DamageRequest estandarizan el daño.





CONCLUSIONES

Flujo Integrado y Solidez Técnica



Desacoplamiento total

Los sistemas operan de forma modular.



Separación de responsabilidades



Robustez en ejecución

Colas y corrutinas coordinadas aseguran consistencia operativa, evitando *race conditions* ante cambios drásticos de estado.



La base técnica está sólida y la arquitectura está limpia... **se viene, indiscutiblemente, el mejor juego de todo el curso.**